

OptimOps technical test: report

Muhammad Safi Ullah Adam, 8 October 2026

Context. A fire and rescue department wants to know where to add up to 12 vehicles so that people are reached faster and fewer calls go unanswered. Using one year of its activity (55,000 calls, 30 stations, 120 communes) and a simulator of its operations, I first checked the official figures, then searched for the best reinforcement plans, and finally audited an analysis produced by an AI assistant.

Setup. To save time during task, I prepared a conda environment (optimops) beforehand with the usual tools: Python, pandas, NumPy, SciPy, Matplotlib, Seaborn and Jupyter. All analysis code was written during the session.

ACT 1. REPRODUCING THE BASELINE

I recalculated the three official indicators from the raw files, following the business guide to the letter. Each call is judged on the first vehicle to arrive. A call that no vehicle could answer counts as 60 minutes, and zone Z4 is left out of the on-time rate because it has no deadline.

Indicator	Official	Recalculated
Time within which 90% of medical emergencies are reached (min)	25.51	25.51
Fires with no fire engine available	290	290
Calls reached on time (%)	86.45	86.46

Two values match exactly. The third differs by 0.01 point, about 5 calls out of 53,525. Arrival times are rounded to the second, and seven calls arrived within one second of their deadline. The more precise duration columns give exactly 86.45, so the gap comes from rounding, not from the method.

One trap: the file has no dispatch order, and in 58% of multi-vehicle calls the first row is not the first vehicle to arrive. I therefore always used arrival times.

What the figures show. Late arrivals sit on the edges of the department, in communes first served by volunteer crews. These areas are late about a third of the time at every hour, against 1 to 6% elsewhere. Afternoons are the hardest, and calls nobody could answer peak from June to August.

First station per commune. I used the official deployment plan (rank 1). For fires, it is the first station on that list with a fire engine, since 14 volunteer stations have none. The data confirms it: rank 1 is the station that most often arrives first in all 120 communes (117 for fires), while "nearest station" fits only 85.

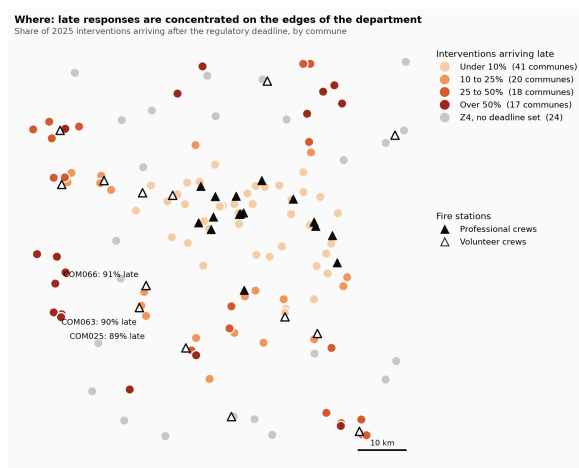


Figure 1. Darker dots are late more often: the problem sits on the edges, far from the professional stations.

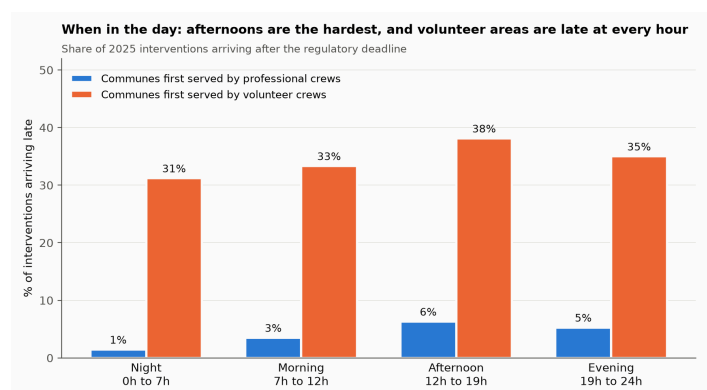


Figure 2. At every time of day, volunteer-served communes are late far more often; afternoons are the worst.

ACT 2. WHERE TO PLACE THE REINFORCEMENT

The service can buy up to 12 vehicles: VSAV (ambulances) or FPT (fire engines). Three goals pull against each other: reach medical emergencies faster; leave fewer fires unanswered, and buy as few vehicles as possible. Each plan is judged by the simulator, which replays a plausible year and never gives the same answer twice.

How I searched, and why. The approach follows the spirit of Cerna Năhais (2022, FEMTO-ST), who paired a simulator of fire-service workload with optimisation. Here every replay is costly (2000 for the whole test) and noisy, so I chose a careful step-by-step search rather than a large genetic algorithm.

1. **Check the simulator.** With no extra vehicle it closely reproduces the 2025 baseline: 25.6 min and about 292 fires, against 25.51 and 290. One replay varies by about 0.2 min and 15 fires, so every result here is an average of 10 to 20 replays.
2. **Split the problem.** Ambulances only change medical response times; fire engines only change unanswered fires. I tested this three times. It turns one hard three-goal problem into two simple ones.
3. **Add one vehicle at a time** where it helps most, then replay each step again with fresh runs, because a search always flatters its own picks. This caught results up to 0.4 min and 8 fires too optimistic.
4. **Build the Pareto front** from the two curves: every split of up to 12 vehicles. It is a menu, not a choice: at equal cost, the service trades minutes against fires.

This used 950 replays, keeping 600 for Act 3.

Plan	Where	Medical P90	Unanswered fires	Who benefits (2025 data)
Today		25.6 min	292	
A, 2 vehicles	2 FPT at CIS14	25.6 min	59 (-80%)	15 communes (Z3, Z4) that had 125 of the 290 unanswered fires
B, 4 vehicles	A + 1 FPT at CIS18 and CIS22	25.6 min	19 (-94%)	41 communes that had 145 of the unanswered fires
C, 8 vehicles	B + 4 VSAV at CIS08, 10, 22, 23	23.2 min	19	B, plus 40 communes with 1,784 medical calls over 20 min
D, 12 vehicles	C + 4 VSAV at CIS05, 09, 17, 19	22.0 min	19	B, plus 86 communes with 4,158 medical calls over 20 min

Recommendation. Buy fire engines first: the first one saves about 170 fires a year; the first ambulance only 0.7 minute. The plans build on each other, so the service can buy in stages. Plan B is the best value; C and D then buy minutes for medical calls. The gains go mostly to the outlying communes that were late in Act 1. The "who benefits" column is read from each plan and the 2025 data, since the simulator only gives department totals.

Limits. These are good plans, not proven optimal ones: the step-by-step search is a heuristic, and after about 4 FPT and 8 VSAV several station choices are within simulation noise. Vehicles count as equal, though an FPT costs more and needs a bigger crew. The simulator gives department totals only, and it replays a year like 2025, not a forecast.

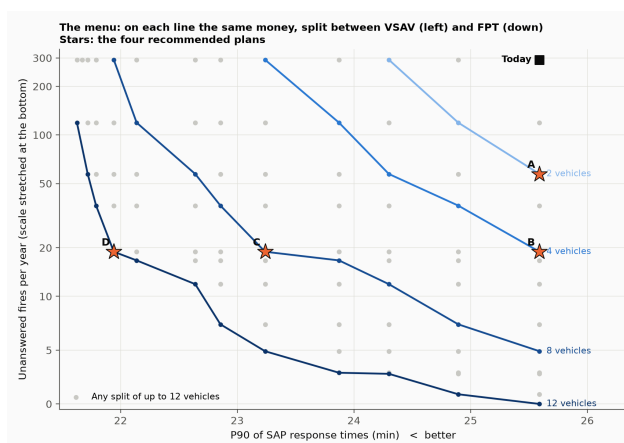


Figure 3. Each line is one budget: moving along it trades medical minutes for fires; stars are plans A to D.

What each plan buys, against today (mean of 20 replays of the whole plan)

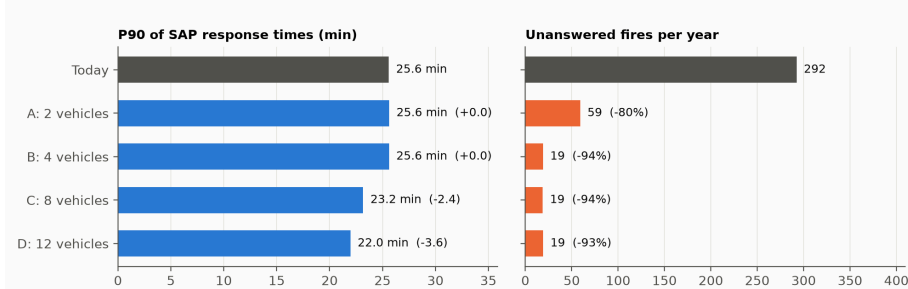


Figure 4. What each plan buys: fire engines first cut unanswered fires by 94%, then ambulances cut 3.6 minutes.

ACT 3. AUDITING THE DELIVERABLE

The script runs and draws a convincing figure, but it contains six errors. I re-ran its own 40 random plans (same seed, 240 replays), fixed the errors one at a time and watched the recommendation. The corrected version is `analysis/audit_corrected.py`.

Error	What it changes in the recommendation
Picks the plan with the best medical time, ignoring fires and cost	Reverses the decision. It recommends 12 vehicles that still leave 192 fires unanswered, while 2 well-placed vehicles on its own list leave 120.
Counts more vehicles as better in the Pareto test	Same pick, but every plan under 8 vehicles disappears, so the service never sees the cheap options.
One replay per plan on a noisy simulator	Same pick here, but results move by up to 37 fires and 7 plans change status on the front.
Drops unanswered calls, works per vehicle row, uses <code>percentile(0.9)</code>	No effect on the pick, but it reports a medical P90 of 2.1 min instead of 25.5.

The lesson is that errors are not equal: three are cosmetic, one reverses the decision. For the same 12 vehicles, plan D from Act 2 gives 22.0 min and 19 fires, against 22.8 min and 192 for the script's choice.

Budget: 1,195 replays used in total. The simulator log shows 1,190: 5 were charged by a run interrupted before it was logged. The log is sent unchanged.

STEERING THE ASSISTANCE

I used the Claude web app to translate the French documents and Claude Code to help with the code. For Act 3, with little time left and the brief asking only for a few clear lines on it, I relied on Claude Code more extensively, so that I could give Acts 1 and 2 most of my own attention. I also took its help to write this report. What I refused or corrected:

- **A heavy setup.** It started installing a 500 MB environment with extra tools. I stopped it and kept only what the task needed.
- **An extra filter.** It proposed keeping only ambulances and fire engines before finding the first arrival. I followed the guide's wording instead (first vehicle to arrive, any type). A check showed both give the same result.
- **Forcing an exact match.** Switching the delay source would give 86.45 exactly, but break the P90. I kept the guide's definition and explained the 0.01 gap instead of fitting the answer.
- **Charts.** It proposed replacing the map with a top-10 bar chart. I kept a simplified map, since the question is "where", but accepted simpler bars for the time of day.
- **A stopping rule.** Its search stopped adding fire engines below 10 fires. Fresh replays showed that 9 was really 17, so I removed the rule.
- **Overclaiming.** I rejected wording such as "optimal plans" and a simulator that "matches" 2025, and asked for the thesis to be cited as the same family of methods, not the same problem.

I accepted, after checking on the data, its suggestion of a separate fire column in the commune table, since 14 stations have no fire engine.

WITH ONE MORE DAY

- Rebuild vehicle availability. Use alert time plus travel and an estimated time on scene. Then keep only calls where every vehicle of the commune's stations was free, and re-test the first-station table on those calls alone.
- Test my own shortcut. Run NSGA-III with the simulator as the oracle on the same 2,000-replay budget, and compare its front with my one-vehicle-at-a-time result. This shows whether the shortcut missed better plans.
- Make the plans robust, not just good on average. Rank plans by their worst years (e.g. the 90th percentile over replays), not only by their mean.
- Test a busier year. Raise summer call volume by 10 to 20% and check that plans A to D keep their order. If they do, the recommendation holds up.
- Use real costs and say who benefits. Count costs in euros and crew members, and report results commune by commune. That would turn the "who benefits" answer from my inference into a measurement.